

Rapport Complet - Forge: Gestion de Projets

Guide détaillé pour débuter avec Next.js

Version: 1.0

Date: Avril 2026

Auteur: Claude IA

Statut: Production-Ready 



Table des matières

- Introduction et contexte
 - Architecture générale
 - Dépendances et leurs rôles
 - Structure des fichiers
 - Concepts clés de Next.js
 - Authentication et sécurité
 - Gestion de l'état
 - API et données
 - Composants UI
 - Styles et thèmes
 - Routes et navigation
 - Guide de développement
-



Introduction et contexte

{#introduction}

Qu'est-ce que ce projet ?

Forge est une application web de gestion de projets permettant aux équipes de :

- Créer et gérer des projets
- Assigner et suivre des tâches
- Collaborer via chat et documents
- Visualiser planification (Gantt, PERT, Kanban)
- Suivre le temps travaillé
- Utiliser un assistant IA

Pourquoi Next.js ?

Next.js est un framework React qui facilite :

- **Routing automatique** : les fichiers deviennent des routes
- **Server-Side Rendering** : meilleure SEO et performance
- **API routes** : backend intégré
- **Middleware** : sécurité et redirection
- **Optimisation** : images, code splitting automatique

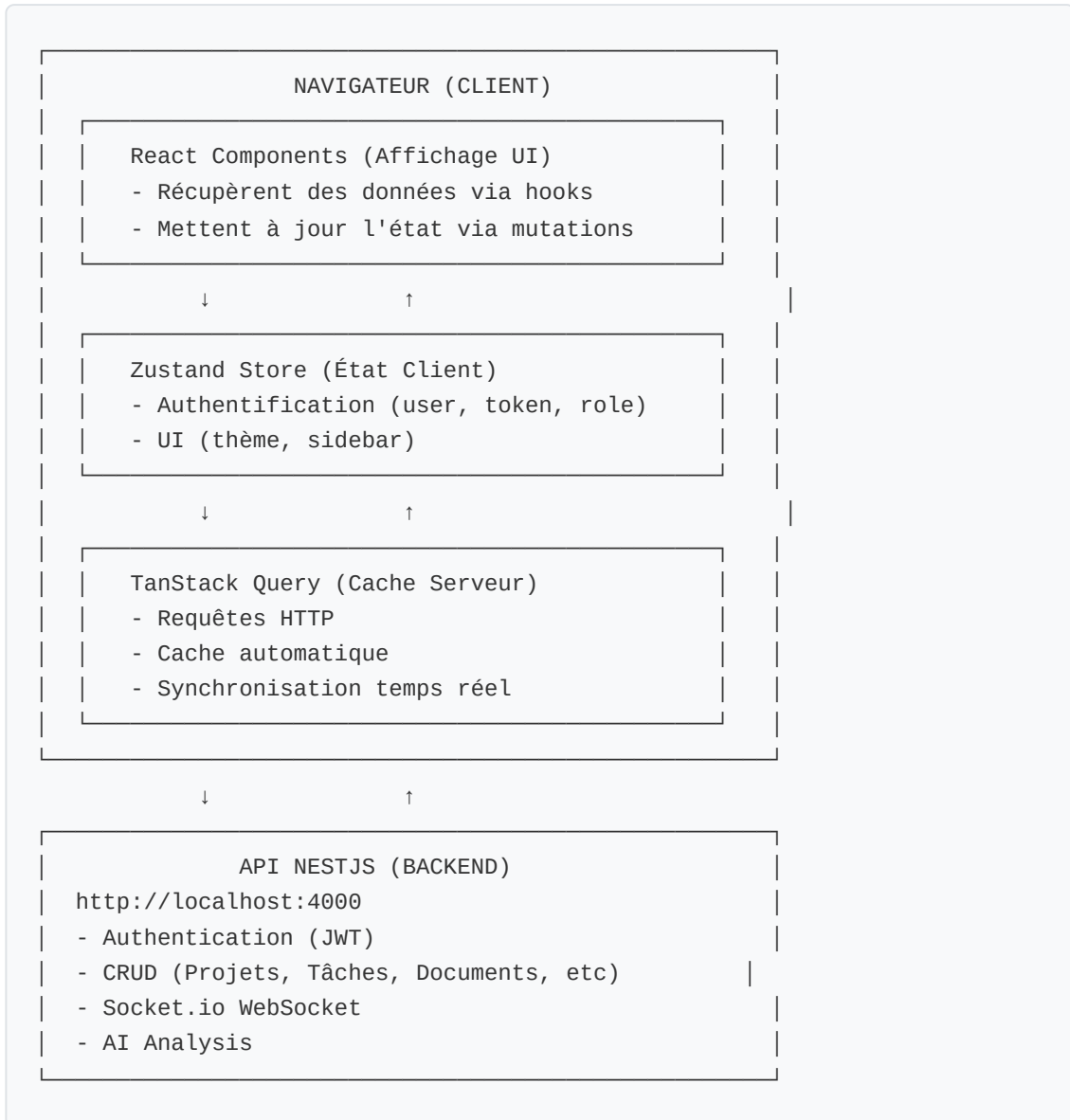
Stack Technologique

Frontend:	Next.js 15 + React 19 + TypeScript
State Management:	Zustand (client) + TanStack Query (serveur)
Styles:	Tailwind CSS + Variables CSS
API:	Axios + Interceptors JWT
Temps Réel:	Socket.io-client
Backend:	NestJS (API externe port 4000)



Architecture générale {#architecture}

Vue d'ensemble



Flux de données

1. **Utilisateur** clique sur un bouton
2. **Composant React** appelle un hook (ex: `useCreateTask()`)
3. **Hook** déclenche une mutation via TanStack Query
4. **Axios** envoie une requête HTTP au backend
5. **Backend** traite et répond
6. **TanStack Query** met en cache et invalide les données liées

7. **Zustand** met à jour l'état si nécessaire (auth, UI)
 8. **React** re-render avec les nouvelles données
 9. **Socket.io** notifie les autres clients en temps réel
-



Dépendances et leurs rôles

{#dépendances}

Dépendances principales

1. next (^15.0.0)

Framework React avec :

- App Router (routing basé sur fichiers)
- Optimisations automatiques
- API routes (middleware.ts)

Rôle: Fondation de l'application

```
// Exemple: Un fichier devient automatiquement une route
// app/(dashboard)/projects/page.tsx → /projects
```

2. react & react-dom (^19.0.0)

Bibliothèque UI et rendering

Rôle: Affichage et interaction

```
// Composant React
export default function ProjectsPage() {
  return <div>Mes projets</div>
}
```

3. typescript (^5.0.0)

Langage typé pour JavaScript

Rôle: Détection d'erreurs avant l'exécution

```
// TypeScript détecte l'erreur
interface User {
  id: string
  name: string
}

const user: User = { id: "1" } // ❌ Erreur: name manquant
```

4. @tanstack/react-query (^5.0.0)

Gestion du cache serveur et requêtes HTTP

Rôle: Synchronisation données serveur ↔ client

```
// Hook pour récupérer et cacher les projets
export function useProjects() {
  return useQuery({
    queryKey: ['projects'],
    queryFn: () => projectsApi.getProjects(),
    staleTime: 5 * 60 * 1000, // Cache 5 min
  })
}
```

Concept clé: Le cache invalide automatiquement après une mutation

5. zustand (^5.0.0)

Gestion de l'état client (authentification, UI)

Rôle: Partager l'état entre composants sans props drilling

```
// Store Auth
export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  token: null,
  login: (token, user) => set({ token, user }),
  logout: () => set({ user: null, token: null }),
}))

// Utilisation dans un composant
function Header() {
  const user = useAuthStore((state) => state.user)
  return <div>{user?.firstName}</div>
}
```

Différence avec Query:

- **Zustand** = État local (auth, UI)
- **Query** = Données serveur (projets, tâches)

6. axios (^1.6.0)

Client HTTP pour les requêtes API

Rôle: Communiquer avec le backend

```
// Instance avec interceptors JWT
const api = axios.create({ baseURL: 'http://localhost:4000' })

api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// En cas d'erreur 401, logout automatique
api.interceptors.response.use(null, (error) => {
  if (error.response?.status === 401) {
    useAuthStore.getState().logout()
    window.location.href = '/login'
  }
  return Promise.reject(error)
})
```

7. react-hook-form + zod

Formulaires et validation

Rôle: Gestion des formulaires avec validation

```
// Validation de formulaire
const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
})

type LoginForm = z.infer<typeof schema>

function LoginForm() {
  const { register, handleSubmit } = useForm<LoginForm>({ resolver: zodResolver(schema) })

  return (
    <form onSubmit={handleSubmit((data) => {
      // data est validé et typé
      loginMutation.mutate(data)
    })}>
      <input {...register('email')} />
      <button type="submit">Login</button>
    </form>
  )
}
```

8. socket.io-client (^4.7.0)

WebSocket pour temps réel

Rôle: Notifications et chat en temps réel

```
// Singleton Socket
import io from 'socket.io-client'

const socket = io('http://localhost:4000', {
  auth: { token: useAuthStore.getState().token }
})

// Écouter les nouveaux messages
socket.on('message:new', (message) => {
  // Invalider le cache pour refetch les messages
  queryClient.invalidateQueries({ queryKey: ['messages'] })
})
```

9. tailwindcss (^3.4.0)

Framework CSS utilitaire

Rôle: Styles avec classes pré-définies

```
// Au lieu d'écrire du CSS, utiliser des classes Tailwind
<div className="flex items-center justify-between p-4 bg-blue-500 text-white">
  Contenu
</div>
```

10. next/font/google

Polices Google optimisées

Rôle: Charger les polices efficacement

```
// Dans layout.tsx
import { Geist, Geist_Mono } from 'next/font/google'

const geistSans = Geist({ variable: '--font-geist-sans', subsets: ['latin'] })
```



Structure des fichiers {#structure}

Organisation du projet


```

forge/
├─ app/                                # App Router Next.js
│   ├─ (auth)/                         # Routes publiques (groupe)
│   │   ├─ login/page.tsx             # Connexion
│   │   ├─ register/page.tsx          # Inscription
│   │   └─ reset-password/page.tsx     # Réinitialiser mot de passe
│   │
│   ├─ (dashboard)/                   # Routes protégées (avec sidebar)
│   │   ├─ layout.tsx                 # Layout avec Header + Sidebar
│   │   ├─ dashboard/                 # Tableau de bord
│   │   │   ├─ page.tsx               # Redirection par rôle
│   │   │   ├─ admin/page.tsx         # Vue Admin
│   │   │   ├─ project-manager/       # Vue Manager
│   │   │   └─ employee/page.tsx      # Vue Employee
│   │   │
│   │   ├─ projects/                 # Pages projets
│   │   │   ├─ page.tsx               # Liste projets
│   │   │   └─ [id]/                 # Projet dynamique
│   │   │       ├─ layout.tsx         # Onglets projet
│   │   │       ├─ kanban/           # Vue Kanban
│   │   │       ├─ gantt/            # Vue Gantt
│   │   │       ├─ pert/             # Vue PERT
│   │   │       ├─ chat/             # Chat temps réel
│   │   │       └─ documents/        # Documents versionnés
│   │   │
│   │   ├─ my-tasks/page.tsx          # Mes tâches
│   │   ├─ time-tracking/page.tsx     # Suivi du temps
│   │   ├─ ai/page.tsx               # Assistant IA
│   │   │
│   │   └─ settings/                 # Paramètres
│   │       ├─ profile/page.tsx       # Mon profil
│   │       ├─ notifications/         # Notifications
│   │       └─ company/page.tsx       # Paramètres entreprise (ADMIN)
│   │
│   ├─ layout.tsx                     # Root layout (HTML, fonts, providers)
│   ├─ page.tsx                       # Page d'accueil
│   └─ globals.css                     # Variables CSS globales + thèmes
├─ middleware.ts                       # Authentification + redirection par rôle
├─ components/                         # Composants réutilisables
│   ├─ ui/                            # Composants de base
│   │   ├─ Button.tsx                # Bouton avec variants
│   │   ├─ Input.tsx                 # Champ texte
│   │   ├─ Modal.tsx                 # Boîte de dialogue
│   │   ├─ Card.tsx                  # Conteneur
│   │   ├─ Alert.tsx                 # Notification
│   │   ├─ Badge.tsx                 # Badge de statut
│   │   ├─ Spinner.tsx               # Loader
│   │   └─ Tooltip.tsx               # Tooltip accessible

```

```

└─ layout/                                # Composants layout
  └─ Header.tsx                          # En-tête avec theme toggle
  └─ Sidebar.tsx                         # Barre latérale navigation
  └─ NotificationBell.tsx                # Cloche notifications
  └─ DashboardLayout.tsx                 # Layout dashboard

└─ lib/                                  # Logique métier (agnostique React)
  └─ api/                                # API clients
    └─ client.ts                         # Instance axios + interceptors
    └─ auth.api.ts                       # Endpoints auth
    └─ projects.api.ts                   # Endpoints projets
    └─ tasks.api.ts                      # Endpoints tâches
    └─ documents.api.ts                  # Endpoints documents
    └─ time-entries.api.ts                # Endpoints temps
    └─ chat.api.ts                       # Endpoints chat
    └─ ai.api.ts                         # Endpoints IA
    └─ company-settings.api.ts           # Endpoints paramètres

  └─ hooks/                              # Hooks TanStack Query
    └─ index.ts                          # Exports centralisés
    └─ useAuth.ts                        # Login, register, profile
    └─ useProjects.ts                    # CRUD projets
    └─ useTasks.ts                       # CRUD tâches
    └─ useChat.ts                        # Chat temps réel
    └─ useDocuments.ts                   # CRUD documents
    └─ useTimeTracking.ts                 # Suivi temps
    └─ useNotifications.ts               # Notifications
    └─ useAI.ts                          # Assistant IA
    └─ useCompanySettings.ts             # Paramètres entreprise

  └─ stores/                             # Zustand stores
    └─ auth.store.ts                     # { user, token, role, login(), logout() }
    └─ ui.store.ts                       # { theme, sidebarOpen, setTheme() }

  └─ types/                              # Types TypeScript (miroir backend)
    └─ user.types.ts                     # User, Role, Department
    └─ project.types.ts                   # Project, ProjectStatus
    └─ task.types.ts                      # Task, TaskStatus, Priority
    └─ planning.types.ts                  # GanttData, PertNode, etc.

  └─ socket/                             # Socket.io client
    └─ socket.client.ts                  # Singleton socket connexion

  └─ utils/                              # Fonctions utilitaires
    └─ api-error.ts                      # Extraction messages d'erreur
    └─ date.ts                           # Formatage dates
    └─ priority.ts                       # Labels + couleurs priorités

└─ public/                              # Assets statiques
  └─ (images, favicon, etc.)

```

├─ CLAUDE.md	# Instructions du projet (backend API)
├─ PLAN_DEV.md	# Plan détaillé du développement
├─ POLISH.md	# Polissage et thèmes
├─ RAPPORT_COMPLET.md	# Ce document
└─ package.json	# Dépendances du projet

Explications des dossiers clés

app/ - App Router (Next.js 15)

Chaque fichier/dossier devient une route :

```
app/login/page.tsx          → GET /login
app/dashboard/projects/page.tsx → GET /dashboard/projects
app/projects/[id]/kanban/page.tsx → GET /projects/123/kanban (dynamic)
```

Les groupes `(auth)` et `(dashboard)` n'affectent pas l'URL :

```
app/(auth)/login/page.tsx → GET /login (pas d'URL /(auth)/)
app/(dashboard)/projects/page.tsx → GET /projects (pas d'URL /(dashboard)/)
```

lib/api/ - API Clients

Chaque fichier communique avec une partie du backend :

```
// lib/api/projects.api.ts
export const projectsApi = {
  getProjects: async () => api.get('/projects'),
  getProjectById: async (id) => api.get(`/projects/${id}`),
  createProject: async (data) => api.post('/projects', data),
  // ...
}

// Utilisation dans un hook
export function useProjects() {
  return useQuery({
    queryFn: () => projectsApi.getProjects(),
  })
}
```

lib/hooks/ - Hooks TanStack Query

Encapsulent la logique data fetch :

```

// Fetch simple (Query)
export function useProjects() {
  return useQuery({ queryKey: ['projects'], queryFn: () => api.get('/projects') })
}

// Mutation (POST/PUT/DELETE)
export function useCreateProject() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: (data) => api.post('/projects', data),
    onSuccess: () => {
      // Invalider le cache pour refetch
      queryClient.invalidateQueries({ queryKey: ['projects'] })
    }
  })
}

```

lib/stores/ - Zustand

État global accessible partout :

```

// lib/stores/auth.store.ts
export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  token: null,
  role: null,

  login: (token, user) => {
    set({ token, user, role: user.role })
    localStorage.setItem('auth-token', token) // Persistance
  },

  logout: () => set({ user: null, token: null, role: null }),
}))

// Utilisation
function MyComponent() {
  const user = useAuthStore((state) => state.user) // Lecture sélective
  const { login } = useAuthStore((state) => state) // Actions
}

```

components/ui/ - Composants réutilisables

Construit en Tailwind, acceptent des props :

```
// components/ui/Button.tsx
interface ButtonProps {
  variant?: 'primary' | 'secondary' | 'danger'
  size?: 'sm' | 'md' | 'lg'
  isLoading?: boolean
  onClick?: () => void
  children: React.ReactNode
}

export default function Button({
  variant = 'primary',
  size = 'md',
  isLoading = false,
  onClick,
  children,
}: ButtonProps) {
  return (
    <button
      className={`
        px-4 py-2 rounded-lg font-medium
        ${variant === 'primary' ? 'bg-blue-500 text-white' : '...'}
        ${size === 'sm' ? 'text-sm' : '...'}
        ${isLoading ? 'opacity-50 cursor-not-allowed' : ''}
      `}
      onClick={onClick}
      disabled={isLoading}
    >
      {isLoading ? <Spinner /> : children}
    </button>
  )
}

// Utilisation
<Button variant="primary" onClick={handleSave}>
  Sauvegarder
</Button>
```

Concepts clés de Next.js {#concepts-nextjs}

1. App Router (vs Pages Router ancien)

✓ Nouveau (App Router - Next.js 13+)
app/dashboard/page.tsx → /dashboard

✗ Ancien (Pages Router)
pages/dashboard.tsx → /dashboard

2. Groupes de routes (name)

Les parenthèses groupent les routes sans affecter l'URL :

```
app/(auth)/login/page.tsx    → /login    (pas /(auth)/login)
app/(auth)/register/page.tsx → /register
app/(dashboard)/projects/page.tsx → /projects (pas /(dashboard)/projects)
```

Utilité: Appliquer des layouts différents

```
// app/(auth)/layout.tsx - Layout login simple
export default function AuthLayout({ children }) {
  return <div className="min-h-screen flex items-center justify-center">{children}</div>
}

// app/(dashboard)/layout.tsx - Layout avec sidebar
export default function DashboardLayout({ children }) {
  return <div className="flex"><Sidebar /><main>{children}</main></div>
}
```

3. Routes dynamiques [param]

```
app/projects/[id]/page.tsx

// Accès aux paramètres
import { useParams } from 'next/navigation'

export default function ProjectPage() {
  const { id } = useParams() // id = "123" si URL = /projects/123
  return <div>{id}</div>
}
```

4. Middleware - Authentication

```
// middleware.ts - S'exécute avant chaque requête
import { NextRequest, NextResponse } from 'next/server'

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value

  // Si pas de token et accès /dashboard → rediriger /login
  if (!token && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url))
  }

  // Vérifier le rôle pour les routes admin
  if (request.nextUrl.pathname === '/settings/company') {
    const role = JSON.parse(request.cookies.get('user-role')?.value || '{}')
    if (role !== 'ADMIN') {
      return NextResponse.redirect(new URL('/dashboard', request.url))
    }
  }

  return NextResponse.next()
}

export const config = {
  matcher: ['/dashboard/:path*', '/settings/:path*', '/projects/:path*']
}
```

5. API Routes (alternative pour backend)

```
// app/api/hello/route.ts
export async function GET(request: Request) {
  return Response.json({ message: 'Hello' })
}

export async function POST(request: Request) {
  const data = await request.json()
  return Response.json({ received: data })
}
```

Note: Ce projet utilise un backend NestJS externe, pas les API routes.

6. 'use client' et Server Components

```
// ✅ Server Component (défaut) - Sûr, pas de js côté client
export default function HomePage() {
  const secretKey = process.env.SECRET // Accessible
  return <div>Contenu server</div>
}

// ❌ Erreur! Hooks dans un Server Component
export default function HomePage() {
  const [count, setCount] = useState(0) // ❌ Erreur!
}

// ✅ Client Component - Permet les hooks
'use client'

export default function Counter() {
  const [count, setCount] = useState(0) // ✅ OK
  return <button onClick={() => setCount(count + 1)}>{count}</button>
}
```

Règle: Utiliser `'use client'` si vous avez besoin de hooks, événements, ou localStorage



Authentication et sécurité

{#authentication}

Flux d'authentification

1. Utilisateur entre email + password
↓
2. Envoyer POST /auth/login au backend
↓
3. Backend retourne { user, access_token }
↓
4. Stocker token en localStorage (Zustand persist)
↓
5. Stocker token en cookie HTTP-only (middleware.ts)
↓
6. Ajouter "Authorization: Bearer TOKEN" à chaque requête (axios interceptor)
↓
7. Si 401 → logout automatique et redirection /login

Implémentation

```

// lib/stores/auth.store.ts
export const useAuthStore = create<AuthStore>()(
  persist(
    (set, get) => ({
      user: null,
      token: null,
      role: null,

      login: (token: string, user: User) => {
        set({ token, user, role: user.role })

        // Créer un cookie HTTP-only (sécurisé)
        if (typeof window !== 'undefined') {
          const thirtyDays = 30 * 24 * 60 * 60
          document.cookie = `auth-token=${token}; path=/; max-age=${thirtyDays}`
        }
      },

      logout: () => {
        set({ user: null, token: null, role: null })
        // Supprimer le cookie
        document.cookie = 'auth-token=; path=/; max-age=0'
      },
    }),
    {
      name: 'auth-store', // localStorage key
      partialize: (state) => ({ user: state.user, token: state.token }),
    }
  )
)

// lib/api/client.ts
const api = axios.create({ baseURL: process.env.NEXT_PUBLIC_API_URL })

// Ajouter le token à chaque requête
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

// Gérer les erreurs 401
api.interceptors.response.use(null, (error) => {
  if (error.response?.status === 401) {
    useAuthStore.getState().logout()
    window.location.href = '/login'
  }
})

```

```
return Promise.reject(error)
})
```

Middleware de protection

```
// middleware.ts
import { NextRequest, NextResponse } from 'next/server'

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value
  const pathname = request.nextUrl.pathname

  // Routes publiques (pas de protection)
  const publicRoutes = ['/login', '/register', '/reset-password']
  if (publicRoutes.some(route => pathname.startsWith(route))) {
    return NextResponse.next()
  }

  // Routes protégées
  if (!token) {
    return NextResponse.redirect(new URL('/login', request.url))
  }

  // Vérifier les rôles
  if (pathname === '/settings/company') {
    // Récupérer le rôle depuis le JWT ou cookie
    // Si EMPLOYEE → rediriger
    return NextResponse.next()
  }

  return NextResponse.next()
}

export const config = {
  matcher: ['/dashboard/:path*', '/projects/:path*', '/settings/:path*']
}
```



Gestion de l'état {#état}

Zustand vs TanStack Query

Aspect	Zustand	TanStack Query
Usage	État client (user, UI)	Données serveur
Persistence	localStorage	Cache en mémoire
Synchronisation	Manuel	Automatique
TTL (Time To Live)	Infini	Configurable (staleTime)
Exemple	<code>user</code> , <code>theme</code>	<code>projects</code> , <code>tasks</code>

Exemple complet

```

// Store client
const useAuthStore = create((set) => ({
  user: null,
  setUser: (user) => set({ user }),
}))

// Hook serveur
const useProjects = () => {
  return useQuery({
    queryKey: ['projects'],
    queryFn: () => projectsApi.getProjects(),
    staleTime: 5 * 60 * 1000, // Cache 5 min
  })
}

// Dans un composant
function Projects() {
  const user = useAuthStore((s) => s.user) // Du store (toujours frais)
  const { data: projects, isLoading } = useProjects() // Du cache (peut être périmé)

  // Si on veut forcer un refresh
  const queryClient = useQueryClient()
  const handleRefresh = () => {
    queryClient.invalidateQueries({ queryKey: ['projects'] })
  }

  return (
    <div>
      <h1>{user?.firstName}'s Projects</h1>
      {isLoading ? <Spinner /> : projects.map(p => <ProjectCard key={p.id} p={p} />)}
      <button onClick={handleRefresh}>Refresh</button>
    </div>
  )
}

```

Invalidation du cache

TanStack Query maintient un cache en mémoire. Quand vous créez/modifiez des données, le cache devient périmé :

```
// Hook pour créer un projet
const useCreateProject = () => {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: (data) => projectsApi.createProject(data),
    onSuccess: (newProject) => {
      // Invalider le cache des projets
      // → TanStack Query refetch la liste automatiquement
      queryClient.invalidateQueries({ queryKey: ['projects'] })
    }
  })
}

// Utilisation
function CreateProjectForm() {
  const mutation = useCreateProject()

  const handleSubmit = (formData) => {
    mutation.mutate(formData) // Créer
    // → onSuccess déclenche invalidate
    // → Les composants utilisant useProjects vont automatiquement refetch
  }
}
```



API et données {#api}

Structure API

```

// lib/api/projects.api.ts
export interface Project {
  id: string
  name: string
  description?: string
  status: ProjectStatus
  createdAt: string
  updatedAt: string
}

export const projectsApi = {
  // GET /projects
  getProjects: async () => {
    const response = await api.get<Project[]>('/projects/my-projects')
    return response.data
  },

  // GET /projects/:id
  getProjectById: async (id: string) => {
    const response = await api.get<Project>(`/projects/${id}`)
    return response.data
  },

  // POST /projects
  createProject: async (data: CreateProjectRequest) => {
    const response = await api.post<Project>('/projects', data)
    return response.data
  },

  // PATCH /projects/:id
  updateProject: async (id: string, data: Partial<Project>) => {
    const response = await api.patch<Project>(`/projects/${id}`, data)
    return response.data
  },

  // DELETE /projects/:id
  deleteProject: async (id: string) => {
    await api.delete(`/projects/${id}`)
  },
}

```

Hooks TanStack Query

```

// lib/hooks/useProjects.ts
export function useProjects() {
  return useQuery({
    queryKey: ['projects'], // Clé unique pour le cache
    queryFn: () => projectsApi.getProjects(),
    staleTime: 5 * 60 * 1000, // Considéré frais pendant 5 min
    gcTime: 10 * 60 * 1000, // Gardé en cache pendant 10 min après inutilisa
  })
}

export function useCreateProject() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: (data: CreateProjectRequest) => projectsApi.createProject(data),
    onSuccess: (newProject) => {
      // Invalider les queries
      queryClient.invalidateQueries({ queryKey: ['projects'] })
    },
    onError: (error) => {
      console.error('Failed to create:', getApiError(error))
    },
  })
}

export function useProjectById(id: string | null) {
  return useQuery({
    queryKey: ['projects', id], // Clé include id → cache différent par projet
    queryFn: () => projectsApi.getProjectById(id!),
    enabled: !!id, // Ne pas fetcher si pas d'id
    staleTime: 10 * 60 * 1000,
  })
}

```

Gestion d'erreurs


```
// lib/utils/api-error.ts
export function getApiError(error: unknown): string {
  if (axios.isAxiosError(error)) {
    return error.response?.data?.message || error.message
  }
  return 'Une erreur est survenue'
}

// Utilisation
const mutation = useCreateProject()

mutation.mutate(data, {
  onError: (error) => {
    const message = getApiError(error)
    alert(message) // "Name is required" au lieu de "400 Bad Request"
  }
})
```



Composants UI {#composants}

Button

```
<Button variant="primary" size="lg" onClick={handleClick} isLoading={loading}>
  Créer
</Button>

// Props disponibles
interface ButtonProps {
  variant?: 'primary' | 'secondary' | 'danger' | 'ghost'
  size?: 'sm' | 'md' | 'lg'
  disabled?: boolean
  isLoading?: boolean
  onClick?: () => void
  children: React.ReactNode
}
```

Input

```
<Input
  label="Email"
  type="email"
  placeholder="user@example.com"
  value={email}
  onChange={(e) => setEmail(e.currentTarget.value)}
  error={emailError}
/>
```

Modal

```
<Modal
  isOpen={showModal}
  title="Créer un projet"
  footer={
    <div className="flex gap-2">
      <Button onClick={() => setShowModal(false)}>Annuler</Button>
      <Button onClick={handleCreate}>Créer</Button>
    </div>
  }
  onClose={() => setShowModal(false)}
>
  <Input label="Nom" value={name} onChange={(e) => setName(e.target.value)} />
</Modal>
```

Card

```
<Card className="p-6 space-y-4">
  <h2>Titre</h2>
  <p>Contenu</p>
</Card>
```

Alert

```
<Alert
  type="error" // 'error' | 'success' | 'warning' | 'info'
  title="Erreur"
  message="Impossible de créer le projet"
  onClose={() => setError(null)}
/>
```



Styles et thèmes {#styles}

Variables CSS

```
/* app/globals.css */
:root[data-theme="light"] {
  --bg-primary: #F8FAFC;
  --bg-surface: #FFFFFF;
  --text-primary: #0F172A;
  --primary: #2F81F7;
  --critical: #F85149;
  --success: #3FB950;
  --warning: #D29922;
}

:root[data-theme="dark"] {
  --bg-primary: #0D1117;
  --bg-surface: #161B22;
  --text-primary: #E6EDF3;
  --primary: #58A6FF;
  /* ... etc */
}
```

Utilisation dans les composants

```
// Méthode 1: Classes Tailwind + var()
<div className="bg-[var(--bg-surface)] text-[var(--text-primary)]">
  Contenu
</div>

// Méthode 2: Directives CSS
<style>{`
  .my-component {
    background-color: var(--bg-surface);
    color: var(--text-primary);
  }
`}</style>
```

Changer le thème

```
// Dans le Header
function Header() {
  const theme = useUIStore((s) => s.theme)
  const setTheme = useUIStore((s) => s.setTheme)

  const toggleTheme = () => {
    const newTheme = theme === 'light' ? 'dark' : 'light'
    setTheme(newTheme) // Applique data-theme sur <html>
  }

  return <button onClick={toggleTheme}>{theme === 'light' ? '🌙' : '☀️'}</button>
}
```

Le thème est persisté dans localStorage et appliqué au chargement :

```
// lib/stores/ui.store.ts
export const useUIStore = create<UIStore>()(
  persist(
    (set) => ({
      theme: 'light',
      setTheme: (theme) => {
        set({ theme })
        document.documentElement.setAttribute('data-theme', theme)
      },
    }),
    { name: 'ui-store' } // localStorage key
  )
)
```



Routes et navigation {#routes}

Structure des routes

Route	Type	Protection	Composant
/	Public	Non	HomePage
/login	Public	Non	LoginPage
/register	Public	Non	RegisterPage
/dashboard	Protected	JWT	DashboardRedirectPage
/dashboard/admin	Protected	JWT + ADMIN	AdminDashboardPage
/projects	Protected	JWT	ProjectsListPage
/projects/[id]/kanban	Protected	JWT	KanbanPage
/my-tasks	Protected	JWT	MyTasksPage
/ai	Protected	JWT	AIAssistantPage
/settings/profile	Protected	JWT	ProfileSettingsPage
/settings/company	Protected	JWT + ADMIN	CompanySettingsPage

Navigation entre pages

```

// Option 1: Composant Link (préfér  )
import Link from 'next/link'

<Link href="/projects/123/kanban">
  Ouvrir le Kanban
</Link>

// Option 2: Hook useRouter (pour redirection)
import { useRouter } from 'next/navigation'

function LoginForm() {
  const router = useRouter()

  const handleLogin = async (credentials) => {
    await loginMutation.mutate(credentials)
    router.push('/dashboard') // Rediriger apr  s login
  }
}

// Option 3: URLs dynamiques
const projectId = '123'
const path = `/projects/${projectId}/kanban`
<Link href={path}>Kanban</Link>

```

Groupes de layout

```

// app/(auth)/layout.tsx - Layout simple pour login
export default function AuthLayout({ children }) {
  return (
    <div className="min-h-screen flex items-center justify-center">
      {children}
    </div>
  )
}

// app/(dashboard)/layout.tsx - Layout avec header + sidebar
export default function DashboardLayout({ children }) {
  return (
    <div>
      <Header />
      <div className="flex">
        <Sidebar />
        <main className="flex-1">{children}</main>
      </div>
    </div>
  )
}

```



Guide de développement {#guide-dev}

Ajouter une nouvelle fonctionnalité

Exemple: Ajouter un bouton "Archiver un projet"

Étape 1: API

```
// lib/api/projects.api.ts
export const projectsApi = {
  // ... autres méthodes ...
  archiveProject: async (id: string) => {
    const response = await api.patch<Project>(`/projects/${id}`, { archived: true })
    return response.data
  },
}
```

Étape 2: Hook

```
// lib/hooks/useProjects.ts
export function useArchiveProject() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: (projectId: string) => projectsApi.archiveProject(projectId),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['projects'] })
    },
  })
}
```

Étape 3: Composant

```

// Dans une page projet
import { useArchiveProject } from '@lib/hooks'

function ProjectDetail({ projectId }) {
  const archiveMutation = useArchiveProject()
  const [confirmDelete, setConfirmDelete] = useState(false)

  const handleArchive = () => {
    archiveMutation.mutate(projectId, {
      onSuccess: () => {
        router.push('/projects') // Rediriger
      },
      onError: (error) => {
        setError(getApiError(error))
      },
    })
  }

  return (
    <div>
      {confirmDelete ? (
        <>
          <span>Êtes-vous sûr?</span>
          <Button onClick={handleArchive} isLoading={archiveMutation.isPending}>
            Confirmer
          </Button>
        </>
      ) : (
        <Button onClick={() => setConfirmDelete(true)}>
          Archiver
        </Button>
      )}
    </div>
  )
}

```

Dépannage courant

Problème: Les données ne se mettent pas à jour après création


```
// ❌ Oublié d'invalider le cache
onSuccess: () => {
  // Les composants ne vont pas refetch!
}

// ✅ Invalider pour refetch automatique
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ['projects'] })
}
```

Problème: Le token n'est pas envoyé aux requêtes

```
// ✅ Vérifier que l'interceptor axios est en place
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

Problème: Erreur "Can't import in non-client component"

```
// ❌ Utiliser useState sans 'use client'
export default function MyComponent() {
  const [count, setCount] = useState(0) // ❌ Erreur!
}

// ✅ Ajouter 'use client'
'use client'

export default function MyComponent() {
  const [count, setCount] = useState(0) // ✅ OK
}
```

Performance

Memoization

```
// Éviter re-renders inutiles
const ProjectCard = React.memo(({ project, onSelect }) => {
  return <div onClick={() => onSelect(project.id)}>{project.name}</div>
})
```

Lazy loading

```
// Importer un composant lourd seulement si utilisé
import dynamic from 'next/dynamic'

const HeavyChart = dynamic(() => import('@components/HeavyChart'), {
  loading: () => <Spinner />,
})

export default function Dashboard() {
  const [showChart, setShowChart] = useState(false)

  return (
    <>
      <button onClick={() => setShowChart(true)}>Afficher graphique</button>
      {showChart && <HeavyChart />}
    </>
  )
}
```

Cache Query

```
// Configurer un bon staleTime
export function useProjects() {
  return useQuery({
    queryKey: ['projects'],
    queryFn: () => projectsApi.getProjects(),
    staleTime: 5 * 60 * 1000, // 5 min = bon équilibre
    gcTime: 10 * 60 * 1000, // Garder 10 min en cache
  })
}
```



Résumé des concepts clés

Concept	Où	Quoi	Pourquoi
Routes	<code>app/</code>	Chaque fichier = route	Routing automatique
Composants	<code>components/</code>	Réutilisables	DRY principe
APIs	<code>lib/api/</code>	Client HTTP	Centraliser les calls
Hooks	<code>lib/hooks/</code>	TanStack Query	Cacher serveur
Stores	<code>lib/stores/</code>	Zustand	État client
Types	<code>lib/types/</code>	Interfaces TS	Type-safety
Middleware	<code>middleware.ts</code>	Auth + rôles	Sécurité
Styles	<code>globals.css</code>	Variables CSS	Thèmes



Pour aller plus loin

Ressources

- [Next.js Documentation](#)
- [TanStack Query](#)
- [Zustand](#)
- [Tailwind CSS](#)

Pratiques recommandées

1. **Toujours typer** avec TypeScript

2. **Utiliser les hooks** pour la logique
3. **Centraliser les APIs** dans `lib/api/`
4. **Invalider le cache** après mutations
5. **Tester les modales** avec des données réelles
6. **Vérifier le responsive** sur mobile

Fin du rapport. Bon développement! 🚀